

Project-Specific Do's, Don'ts, Failures, And Future Work

This is not generic RL advice. It is the postmortem from this Furuta project, based on the project docs and saved results.

Top Lessons To Put On Slides

These are the best high-signal lessons for a short presentation.

1. Do Not Trust Training Reward Alone

What happened:

- Early TD3 stages learned to survive training episodes, but fixed evaluation still failed too often.
- The first MathWorks-style narrow run had 'FailureRate = 0', but mean final theta2 error was still about '30.7 deg'.
- The detailed 1c runs survived full episodes and reached high average rewards, but learned repeated swinging instead of upright capture.

Concrete example:

```

detailed 1c scratch7000:
FailureRate = 0
MeanFinalTheta2MAE = 1.645 rad
MeanTheta2EndPeakToPeak = 6.235 rad
```

Lesson:

Surviving an episode is not the same as balancing the pendulum.

2. Fixed Evaluation Saved The Project More Than Once

What happened:

- Training traces often looked promising.
- Fixed-case evaluation exposed arm-limit failures, loose final balance, persistent oscillation, and later the detailed-model survival loophole.
- The 297-case grid made it possible to compare agents fairly.

Lesson:

Every serious run needs fixed short and full evaluations, not only training reward plots.

3. The Reward Was Often Technically Correct But Practically Weak

What happened:

- In the direct Stage 1 run, the theta2 reward near upright was extremely small.
- A '0.5 deg' pendulum oscillation cost was almost free relative to survival.
- Velocity penalties were also tiny compared with alive/upright bonuses.
- In the detailed model, the alive/survival reward could be exploited by continuous swinging.
- A late observed variant was even more concrete: the agent learned to hold or hover the pendulum around a sideways region with jitter instead of completing swing-up and capture.

Lesson:

Reward terms must be scaled so the behavior we actually care about is visible to the critic.

4. Do Not Confuse Safety Failure Rate With Control Quality

What happened:

- The detailed 1c 7000-episode run had 'FailureRate = 0', but did not balance.
- The actor-small 500 Hz run had slightly fewer full-grid safety failures than the baseline, but worse final theta2 error, higher oscillation, higher action variation, and higher electrical energy.

Lesson:

A lower failure rate can hide worse balance. Always inspect final error, oscillation, action variation, and energy.

5. Observation Design Is A Control Decision

What happened:

- Raw wrapped angle observations were unsuitable for global swing-up because of the wrap discontinuity.
- Switching to sin/cos angle observations was a major improvement.
- The later unsigned arc-distance observation reduced dimension, but removed too much direction information and failed badly.

Lesson:

Compact observations are useful only if they preserve the information needed to choose the control direction.

6. The Critic Can Be The Bottleneck

What happened:

- Actor 1x64 / critic 1x64 failed and barely acted.
- Actor 1x64 / critic 2x64 learned meaningful behavior.
- The smaller actor was deployable-looking, but not better overall for balance.

Lesson:

You can make the deployed actor smaller, but the critic still needs enough capacity to train a useful policy.

7. Match Training, Evaluation, And Deployment Timing

What happened:

- Early TD3 became very slow because learning updates happened too often during long surviving episodes.
- 'LearningFrequency = -1' moved learning to episode end and made the workflow closer to MathWorks examples.
- A later 500 Hz parallel evaluation bug initialized workers with the default 200 Hz config, contaminating old 500 Hz evaluation artifacts.

Lesson:

Sample time and workspace config are part of the experiment. If they are wrong, the comparison is wrong.

500 Hz long reference:

- Baseline run:
'results/TD3/run_20260618_223832_td3_mathworks_style_pi_current_1b_500hz_long'
- Training model: 'inv_rot_pen_RL_cntr_simscape_sim_1b_train'
- Plant sample time: '5e-5 s' / 20 kHz.
- Agent sample time: '0.002 s' / 500 Hz.
- Episode duration: '5 s'.
- Maximum steps per full episode: '5 / 0.002 = 2500' agent steps.
- Planned training length: '5000' episodes.
- Maximum environment interaction:
'5000 * 2500 = 12,500,000' agent steps.

8. Hardware Scaling Is Part Of The Controller

What happened:

- The 500 Hz policy was trained with a 4 A current scale.
- On hardware, full authority caused more strain and jitter.
- Scaling to '0.4 * CurrentScale' gave more stable hardware operation.

Lesson:

If a hardware limiter or scale factor changes the behavior, it should be documented and eventually mirrored in training/evaluation.

9. Fine-Tuning Can Destroy A Working Policy

What happened:

- Near-upright detailed-model fine-tuning from the old '500Hz_long' agent initially improved, then collapsed around episodes '46-55'.
- A smoke test suggested that loaded agent options/exploration/warm-start needed to be reapplied carefully.

Lesson:

When adapting a working policy, use gentle updates, low exploration, clear option overrides, and possibly policy-preserving regularization.

10. Do Not Add Domain Randomization Too Early

What happened:

- After hardware mismatch was observed, domain randomization looked attractive.
- Detailed-model training then showed the nominal detailed model still did not learn clean capture/balance.

Lesson:

Randomizing a nominal model that cannot solve the task just makes a harder unsolved task.

Detailed Mistakes And What We Learned

Mistake 1 - Early DDPG Was Pushed Too Long

What failed:

- DDPG did not reliably learn local upright stabilization.
- It produced high-frequency oscillatory torque around '125 Hz'.
- Fixed post-stage cases kept failing.

What we learned:

- The plant and sampled controller path were controllable, because the classical/state-space controller could stabilize small errors.
- The DDPG actor/critic training was too fragile for this delayed, saturated, unstable system.
- TD3 was a better next step because it changed fewer assumptions than SAC while addressing DDPG overestimation and unstable policy updates.

Do:

- Move on when a method repeatedly fails controlled diagnostics.

Do not:

- Keep retuning DDPG blindly when the algorithmic failure mode is visible.

Mistake 2 - Early Reward Could Prefer Quick Failure Or Weak Survival

What failed:

- The original reward could make quick failure less costly than a longer, unstable survival attempt.
- Later, survival/alive reward could dominate fine behavior and allow continuous swinging.

What we learned:

- Terminal penalties and alive rewards need sane relative scale.
- Very large unsafe penalties can create harsh critic targets.
- Very generous survival reward can create a loophole where staying alive is enough.

Do:

- Inspect reward components numerically.
- Log reward diagnosis signals.
- Ask whether the reward differentiates "excellent" from "barely acceptable."

Do not:

- Assume a reward formula is good because it is mathematically reasonable.

Mistake 3 - We Judged Some Runs Too Broadly Or Too Narrowly At First

What failed:

- Stage 1 direct TD3 was evaluated on a broad 297-case swing-up grid even though it was only trained for near-upright behavior.
- Conversely, some early training reward/survival looked good until fixed grids exposed failures.

What we learned:

- Evaluation must match the training question.
- A local-balance stage needs a local-balance grid.
- A final swing-up controller needs the broad grid.

Do:

- Use both task-specific evaluation and final broad evaluation.

Do not:

- Declare a local stage useless only because it fails the final full task.

Mistake 4 - Frequent In-Episode TD3 Updates Made Training Painful

What failed:

- With 1 kHz agent rate and 'LearningFrequency = 4', a full 1 s episode could trigger about '250' learning events.
- With off-policy mini-batch limits, the option-level upper bound could become very large.
- Training slowed as episodes survived longer.

What we learned:

- Simulink plant simulation was not the main slowdown; learner updates were.
- 'LearningFrequency = 40' reduced pressure.
- 'LearningFrequency = -1' moved learning to episode end and gave a cleaner rollout.

Do:

- Estimate update counts before long runs.

Do not:

- Treat sample time and learning frequency as independent details.

Mistake 4b - We Needed To Understand End-Of-Episode Learning Quantitatively

This became important for the '500Hz_long' run because the controller was fast enough and long enough that "one training run" was no longer a small object.

The key settings from the deployed baseline were:

Run: run_20260618_223832_td3_mathworks_style_pi_current_1b_500hz_long
Agent sample time: 0.002 s / 500 Hz

Plant sample time: 5e-5 s / 20 kHz
Episode duration: 5 s
Max episodes: 5000
LearningFrequency: -1
MiniBatchSize: 1024
NumEpoch: 10
MaxMiniBatchPerEpoch: 100
PolicyUpdateFrequency: 2
ExperienceBufferLength: 2.5e6
CurrentScale: 4 A
CurrentMax: 4 A

What 'LearningFrequency = -1' means:

- MATLAB collects rollout data during the episode.
- It performs the learning phase at the end of the episode.
- This avoids interrupting Simulink rollout with frequent learner updates.
- It also makes the computational cost easier to reason about.

Maximum environment steps:

$\text{MaxEpisodes} * \text{EpisodeDuration} / \text{AgentSampleTime}$
 $= 5000 * 5 / 0.002$
 $= 12,500,000$ agent steps

Maximum minibatch updates:

$\text{MaxEpisodes} * \text{NumEpoch} * \text{MaxMiniBatchPerEpoch}$
 $= 5000 * 10 * 100$
 $= 5,000,000$ minibatch updates

This is an upper bound, not a guaranteed exact count. The actual number can be lower if:

- the replay buffer is not ready early in training,
- episodes terminate early,
- the toolbox skips or reduces some updates because of available data,
- the run is interrupted or stopped manually.

Mini-batch size vs maximum number of minibatches:

- 'MiniBatchSize = 1024' is how many stored transitions are sampled for one gradient update.
- 'MaxMiniBatchPerEpoch = 100' limits how many such gradient updates can happen in one epoch.
- 'NumEpoch = 10' repeats that end-of-episode learning pass up to 10 times.

For TD3 specifically:

- the critics are updated at the main update rate,
- 'PolicyUpdateFrequency = 2' means the actor is updated only every second critic-update step,
- this delayed actor update is part of TD3's stability improvement over DDPG.

Training time:

- The exact original '500Hz_long' wall-clock runtime was not documented with a clean start/end diary.
- A closely related '500 Hz long' actor1x64/critic2x64 comparison run was initialized at about '2026-06-29 13:05' and wrote the final agent around '16:41', about '3 h 36 min'.
- Later detailed 500 Hz 5000-episode runs were documented as taking "several hours" of active training, while timestamp-derived durations included idle or waiting time.

Do:

- Quote '500Hz_long' runtime as "several hours" unless a precise diary is found.
- Quote exact update counts as upper bounds.
- Separate environment interaction count from neural-network update count.

Do not:

- Say "5000 episodes" as if that alone describes training cost.
- Confuse 12.5 million agent steps with 5 million minibatch updates.
- Forget that each minibatch update samples 1024 transitions.

Mistake 5 - Raw Wrapped Angles Were A Bad Global Observation

What failed:

- Near-upright raw errors were acceptable locally, but global swing-up has angle wrapping.
- The scalar wrapped 'theta2Error' jumps at the downward position.

What we learned:

- Sin/cos observations avoid exposing the discontinuity to the network.
- The scalar wrapped error can still be used for diagnostics and reward if handled carefully.

Do:

- Use circular representations for global angle tasks.

Do not:

- Feed a discontinuous wrapped scalar into the policy and expect smooth global learning.

Mistake 6 - The First Strong MathWorks-Style Run Still Had Hidden Weaknesses

What failed:

- The narrow MathWorks-style run had zero failures but poor final upright accuracy.
- Survival and safety did not imply tight stabilization.

What we learned:

- The MathWorks-style pattern was useful, but only fixed metrics showed what quality was missing.
- Wider reset and longer training improved the controller.

Do:

- Keep the successful workflow, but evaluate quality beyond survival.

Do not:

- Present "0 failures" as "solved."

Mistake 7 - Direct Voltage Was A Useful Diagnostic But Not A Winner

What failed:

- Direct voltage reduced final oscillation in some easy cases.
- But broad robustness was worse than PI/current.
- Analytical-active versus Simscape-active evaluation diverged strongly.

What we learned:

- The actuator interface and active plant model are separate confounds.
- Simscape is not just visualization if it is active in the feedback loop.

Do:

- Train and evaluate with the same active plant unless intentionally testing transfer.

Do not:

- Mix analytical-active and Simscape-active results as if they are the same environment.

Mistake 8 - Model Logging Assumptions Broke Evaluation

What failed:

- The voltage model logged 'voltage_command' instead of 'current_command'.
- The signal extractor initially expected the current-command name.

What we learned:

- Analysis scripts must understand the actuator interface.
- A generic "actuator command" abstraction is safer than hard-coding one signal name.

Do:

- Make extraction code tolerant to expected model variants.

Do not:

- Let a logging name silently define what can be evaluated.

Mistake 9 - Old 500 Hz Evaluation Was Contaminated By Workspace Config

What failed:

- Parallel evaluation workers initialized with the default config.
- The default config used 200 Hz sample time.
- Some older 500 Hz eval artifacts were therefore sample-time contaminated.

What we learned:

- Parallel workers need the saved run config, not a generic default config.
- The fix was carrying 'evalCfg.WorkspaceConfig = cfg'.

Do:

- Prefer corrected 'full_fixed_workspace_*' evaluations for 500 Hz comparisons.

Do not:

- Trust old 500 Hz parallel eval files without checking the workspace config path.

Mistake 10 - Smaller Network Was Tested In The Right Way, And Failed In A Useful Way

What failed:

- Symmetric actor 1x64 / critic 1x64 failed and barely acted.
- It mostly failed with too little control effort, not violent commands.

What we learned:

- The critic was likely too small.
- Actor 1x64 / critic 2x64 restored learning.
- But smaller actor did not automatically reduce jitter or improve balance.

Do:

- Keep critic capacity when testing a smaller deployed actor.

Do not:

- Equate a smaller network with smoother or safer behavior.

Mistake 11 - Pure Unsigned Arc-Distance Observation Removed Too Much Direction

What failed:

- The reduced observation run reached 1837 episodes but still terminated after tens of steps.
- Last average reward was around '-1073'.

What we learned:

- One scalar can give distance without wrap, but it loses side/direction.

- 'theta1' especially needs sign for arm centering and arm-limit avoidance.

Do:

- Restore at least signed 'theta1' if testing compact observations.

Do not:

- Remove directional angle information just to reduce observation dimension.

Mistake 12 - Detailed Model Training Found A Reward Loophole

What failed:

- Detailed 1c scratch and fine-tune runs survived but did not balance.
- The latest 1c 7000-episode 1.5 A run learned smooth repeated swinging.
- One late-week agent showed a sideways-hold failure mode: it did not learn a clean swing-up, but it could keep the pendulum near a side position with jitter, which the reward treated as preferable to quick failure.

What we learned:

- Detailed dynamics changed the reward landscape.
- The old survival reward was not enough for capture.
- The next reward needs upright capture and damping terms.

Do:

- Fix nominal detailed-model capture before randomization.

Do not:

- Interpret full-length detailed-model episodes as success.

Mistake 13 - Fine-Tuning Was Too Easy To Corrupt

What failed:

- A loaded policy initially looked good on a near-upright detailed-model fine-tune, then collapsed.
- A smoke test revealed that loaded agent TD3 options needed to be overridden from the new config.

What we learned:

- Loading a trained agent also brings old options/noise/warm-start behavior.
- Fine-tuning should reapply options explicitly.
- A working policy should be protected during adaptation.

Do:

- Run zero-exploration smoke tests before fine-tuning.
- Reapply agent options after loading.

Do not:

- Assume the config automatically changes every option inside a loaded agent.

Mistake 14 - Hardware Scaling Was Treated As A Test Patch First

What happened:

- The policy trained with 4 A authority.
- Hardware ran better when action was scaled by '0.4' before the current clamp.

What we learned:

- Scaling changes the effective policy.
- If scale '0.4' is the safer behavior, future training should include that current authority or at least evaluate it explicitly.

Do:

- Document current scaling in every hardware run.

Do not:

- Compare hardware and simulation without matching action scaling and clamps.

Mistake 15 - Domain Randomization Was Tempting Too Early

What happened:

- After hardware mismatch, the natural instinct was to randomize model parameters.
- But the nominal detailed model still did not learn capture.

What we learned:

- Domain randomization is for robustness around a solvable nominal task.
- It is not a substitute for a reward/observation/model that can solve the base task.

Do:

- Add randomization gradually after nominal capture works.

Do not:

- Randomize many parameters at once to hide a known nominal failure.

Why We Did Not Use The Brian Douglas Hybrid SAC/PPO Architecture Directly

The Brian Douglas / MathWorks QUBE Servo2 reference in 'references/' uses a hybrid architecture:

classical feedback controller: upright balance

SAC agent: swing-up behavior / reference

PPO agent: mode selection between swing-up and balance

The reference also runs the RL agents more slowly than many of our early tests:

feedback control period $T_c = 0.005 \text{ s}$ -> 200 Hz

RL sample time $T_s = T_c * 4 = 0.020 \text{ s}$ -> 50 Hz

episode length $T_f = 10 \text{ s}$

RL steps per episode = 500

Why this architecture is attractive:

- classical control handles the region where linearization works well,
- RL only needs to learn the nonlinear swing-up part or switching logic,
- the mode selector separates "when to swing" from "when to balance",
- it is closer to a practical supervised/hybrid controller than pure end-to-end RL.

Why we did not follow it directly:

1. Different project question.

This project intentionally tested a direct learned controller for the Furuta model. Using classical control for balance from the beginning would have made the result more like RL-assisted switching and less like direct RL control.

2. Integration cost.

The Brian Douglas example targets the Quanser QUBE/Raspberry Pi workflow. Our setup has the ZHAW Furuta model, SLDRT, UART, a microcontroller current PI loop, and different hardware signal conventions. Porting the full hybrid architecture would have added a large second integration project.

3. Debugging complexity.

The hybrid system has three controllers to debug: SAC swing-up, PPO mode selector, and classical balance controller. Early on, we were still finding issues in reward scale, action scaling, sample time, observations, logging, active plant choice, and hardware communication. Direct TD3 gave a cleaner baseline.

4. Controlled algorithm progression.

We started from DDPG. TD3 was the closest next step because it keeps the same deterministic continuous-action actor but fixes major DDPG weaknesses with twin critics, delayed actor updates, and target smoothing. SAC/PPO would have changed more assumptions at once.

What we learned from it anyway:

- The hybrid reference strongly supports using sin/cos angle observations.
- It supports slower RL rates with a faster low-level controller underneath.
- It shows that swing-up and balance can be separated into different control tasks.
- It remains a good future architecture if direct TD3 capture stays brittle.

DDPG vs TD3 vs SAC vs PPO

DDPG

DDPG is an off-policy actor-critic method for continuous action spaces.

In plain language:

actor: maps observation -> deterministic action
critic: estimates $Q(\text{observation}, \text{action})$
replay buffer: reuses old transitions for learning

Why it was attractive:

- one continuous motor command,
- actor can be deployed directly,
- simpler than SAC/PPO hybrid architectures.

What happened here:

- DDPG did not reliably learn local upright stabilization,
- fixed post-stage cases failed,
- the actor produced high-frequency oscillatory torque,
- the method was too fragile for our delayed, saturated, unstable setup.

TD3

TD3 is essentially a stronger DDPG-family method.

Key changes:

- two critics reduce over-optimistic Q estimates,
- actor updates are delayed relative to critic updates,
- target action smoothing discourages brittle sharp Q peaks.

Why it became our main method:

- still gives a deterministic actor for deployment,
- closest controlled step after DDPG,
- MathWorks QUBE direct TD3 example gave a useful template,
- produced the first useful Furuta swing-up/stabilization baseline.

Main limitation observed:

- TD3 can still exploit reward loopholes,
- fine-tuning can corrupt a working policy,
- it does not automatically solve sim-to-real mismatch or jitter.

SAC

SAC is Soft Actor-Critic. It is off-policy like TD3, but uses a stochastic actor and an entropy term.

In plain language:

learn high reward while also keeping the policy exploratory/robust

Why it is interesting:

- strong exploration,
- often good for swing-up and contact-rich/nonlinear tasks,
- used in the Brian Douglas hybrid QUBE example for swing-up.

Why we did not switch to it during the main path:

- it would change more assumptions than TD3,
- the policy architecture and training behavior differ more,
- our immediate need after DDPG was a controlled comparison,
- deployment would still need careful deterministic/exploitation handling.

Good future use:

- SAC for a dedicated swing-up policy,
- especially inside a hybrid architecture with classical balance or a supervisor.

PPO

PPO is Proximal Policy Optimization. It is an on-policy policy-gradient method.

In plain language:

collect trajectories with the current policy,
update the policy without changing it too aggressively,
discard old data,
repeat.

It can be used for discrete or continuous actions. In the Brian Douglas hybrid example, PPO is used for the discrete mode-selection policy:

mode 0: use one reference/controller behavior
mode 1: use swing-up output / another behavior

Why it was not our main controller:

- our motor command is continuous current, where TD3/SAC are more direct choices,
- PPO is on-policy and can be sample-hungry,
- the place where PPO naturally fits here is a supervisor/mode selector, not necessarily the low-level current command.

Good future use:

- choose between swing-up, capture, and balance modes,
- choose between RL and classical fallback,
- learn a supervisor around safer lower-level controllers.

Weto Inputs: What Worked And What Did Not

After the meeting with Thomas Weinmann, we tested suggestions as controlled experiments.

Smaller Networks

Suggestion:

try a smaller network, especially because upright balance is locally close to a linear control problem.

Test 1:

actor 1x64
critic 1x64

Result:

- failed,
- did not learn useful swing-up,
- failed while using very little control effort,
- likely underfit or critic too weak.

Test 2:

actor 1x64
critic 2x64

Result:

- learned meaningful behavior,
- confirmed that a small actor can be trained if the critic remains stronger,
- but not better overall than the deployed 500 Hz baseline,
- worse final theta2 error, oscillation, action variation, and electrical energy.

Conclusion:

Small actor is possible, but critic capacity matters. This did not solve the hardware jitter problem by itself.

Reduced Observation / Arc Distance

Suggestion:

reduce observation size by using angle distance around the circle instead of sin/cos pairs.

Test:

theta1_arc_norm
theta2_arc_norm
omega1_scaled_norm
omega2_scaled_norm
previous_action

Result:

- failed/interrupted,
- after 1837 episodes, episodes were still only tens of steps,
- likely removed too much direction information.

Conclusion:

The observation became compact but ambiguous. Future reduced observations should keep at least signed theta1.

State-Dependent Reward Shaping

Suggestion:

make reward terms depend on state; near upright, punish jitter/current/delta-u more strongly than during swing-up.

Status:

- not fully solved yet,
- capture-tune config prepared,
- latest detailed 1c result makes this the most important next direction.

Conclusion:

This is likely more important now than more network-size experiments.

Add Actuator / Controller State

Suggestion:

add measured current, PI output, saturated voltage, or another actuator state to improve the Markov property.

What we tried:

- 1d near-upright scratch with 'I_meas' as an 8th observation.

Result:

- both initial scratch attempts failed fixed eval,
- not enough to prove 'I_meas' is bad,
- likely the scratch setup/reward/timeout was too harsh.

Conclusion:

Actuator state remains plausible, but should be tested with a clearer reward and gentler setup.

Improved Detailed Model

Suggestion:

model encoder quantization, filters, static friction, current-loop behavior, dead-zone compensation, and measured-current noise.

Result:

- detailed 1c/1d model path was built,
- nominal detailed training exposed the current reward loophole,
- it did not immediately improve control, but it clarified the next problem.

Conclusion:

This was the most productive Weto direction. It shifted the project from simple-model success to real sim-to-real bottlenecks.

Sim-To-Real Research Summary

The latest detailed-model results look less like "train longer" and more like a transfer/adaptation problem:

simple model policy works partly on hardware,
detailed model changes dynamics enough that naive fine-tuning can fail,
hardware logs show scaling/friction/current-loop mismatch.

SimOpt / Adaptive Domain Randomization

Paper:

Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience
<https://arxiv.org/abs/1810.05687>

Idea:

- do not guess randomization ranges forever,
- use real-world rollouts to adapt the simulator/randomization distribution.

Relevance:

- hardware logs can tune friction, current scaling, delay, and dead-zone ranges.

Residual Reinforcement Learning

Paper:

Residual Reinforcement Learning for Robot Control
<https://arxiv.org/abs/1812.03201>

Idea:

- keep a useful base controller,
- learn an additive correction.

Relevance:

- the '500Hz_long' policy already has useful behavior,
- residual correction near upright may be safer than full TD3 fine-tuning.

EPOpt / Robust Model Ensembles

Paper:

EPOpt: Learning Robust Neural Network Policies Using Model Ensembles
<https://arxiv.org/abs/1610.01283>

Idea:

- train over sampled model variants,
- emphasize difficult cases.

Relevance:

- useful after nominal detailed capture works,
- could train robustness to friction/current/dead-zone variants.

Multi-Fidelity RL

Paper:

Multi-Fidelity Reinforcement Learning with Gaussian Processes
<https://arxiv.org/abs/1712.06489>

Idea:

- combine cheap/simple models and expensive/accurate models.

Relevance:

- matches our path:

simple analytical model -> detailed hardware-like model -> hardware logs

Meta-RL For Adaptation

Paper:

Learning to Adapt in Dynamic, Real-World Environments Through Meta-Reinforcement Learning
<https://arxiv.org/abs/1803.11347>

Idea:

- train policies that can adapt quickly to changed dynamics.

Relevance:

- less immediate, but relevant if friction/offsets vary over time.

Policy Distillation / Learning Without Forgetting

Papers:

Policy Distillation
<https://arxiv.org/abs/1511.06295>

Learning without Forgetting
<https://arxiv.org/abs/1606.09282>

Idea:

- transfer or fine-tune while preserving useful old behavior.

Relevance:

- our fine-tuning collapse suggests an action-deviation penalty, teacher regularization, or residual policy could help preserve the original '500Hz_long' behavior.

Practical Research Takeaway

For this project:

1. Do not jump straight to broad randomization.
2. Preserve the working simple-model policy.
3. Adapt gently on the detailed model.
4. Use hardware logs to tune mismatch ranges.
5. Add domain randomization only after nominal capture works.

Future Work Recommendations

Immediate Next Step

Continue from:

results/TD3/run_20260708_222715_td3_mathworks_style_pi_current_1c_500hz_long_detailed_scratch7000

Run:

scripts/trainFurutaDetailed1c500HzCaptureTuneFromScratch7000.m

Goal:

Preserve the learned swing-up-like behavior, but make upright capture and damping more valuable than continued swinging.

Reward Direction

Use state-dependent shaping:

far from upright:

allow energy injection

near upright:

reward capture

penalize θ_2 error

penalize ω_2 and ω_1

penalize action/current chatter

Potential additions:

- upright capture bonus,
- local θ_2 cost inside 25 deg,
- local velocity damping,
- near-upright action-difference/current penalty,
- pre-capture arm travel or reversal metrics before reward changes.

Fine-Tuning Direction

Try:

- lower actor/critic learning rates,
- lower exploration noise,
- replay reset when changing model fidelity,
- zero-exploration smoke tests,
- teacher-policy/action-deviation penalty,
- residual policy on top of the baseline.

Observation Direction

Keep the 7-state sin/cos baseline unless testing a specific hypothesis.

If adding actuator state, test one at a time:

- ' l_{meas} ',
- current command,
- saturated PI voltage/output,
- PI/integrator state if exposed.

Detailed Model Direction

Continue using open-loop hardware tests for calibration:

- friction,
- dead-zone/static current compensation,
- current-loop filtering,
- encoder quantization,
- velocity filtering,
- current measurement noise.

Domain Randomization Direction

Add only after nominal detailed capture works.

Recommended phases:

1. θ_1 friction and damping,
2. current scale and current limit,
3. delay/filter/dead-zone compensation,
4. sensor noise and quantization,

5. physical masses/inertias if needed.

Hardware Data Direction

Use hardware logs first for:

- validation,
- mismatch diagnosis,
- action/current scaling checks,
- estimating friction/dead-zone ranges.

Use for residual learning or offline replay only after:

- enable windows are segmented,
- signals are synchronized,
- unsafe or disabled intervals are excluded,
- the reward/isDone semantics are clear.